

SECURE CODING REVIEW OF CREATORAI DASHBOARD

CodeAlpha Cyber Security Internship

Task 3: Secure Coding Review

Prepared By:

Opeyemi Oluwatimilehin Talabi

Submitted To:

CodeAlpha Cyber Security Internship Program

Project Type:

Security Assessment Report

Date:

10 June 2026

1. Introduction

This project presents a secure coding review of CreatorAI Dashboard, a simple AI content creator platform developed for educational purposes. The application simulates common features found in modern creator platforms, including user authentication, thumbnail uploads, and user information handling.

The objective of this review is to identify security weaknesses within the application, assess their potential impact, and recommend appropriate remediation measures based on secure coding principles.

2. Application Overview

CreatorAI Dashboard is a Python-based application that allows users to:

- Log into the platform
- Upload thumbnail files
- Enter email information
- Simulate AI service integration through an API key

The application was intentionally reviewed from a security analyst's perspective to identify vulnerabilities that could affect confidentiality, integrity, and availability.

2.1 Tools Used

- Python 3.14.5
- Windows Command Prompt
- Microsoft Word
- Manual Code Review

2.2 Application Execution Evidence

The screenshot below shows the successful execution of the CreatorAI Dashboard application, including user authentication, thumbnail upload simulation, and email handling functionality.

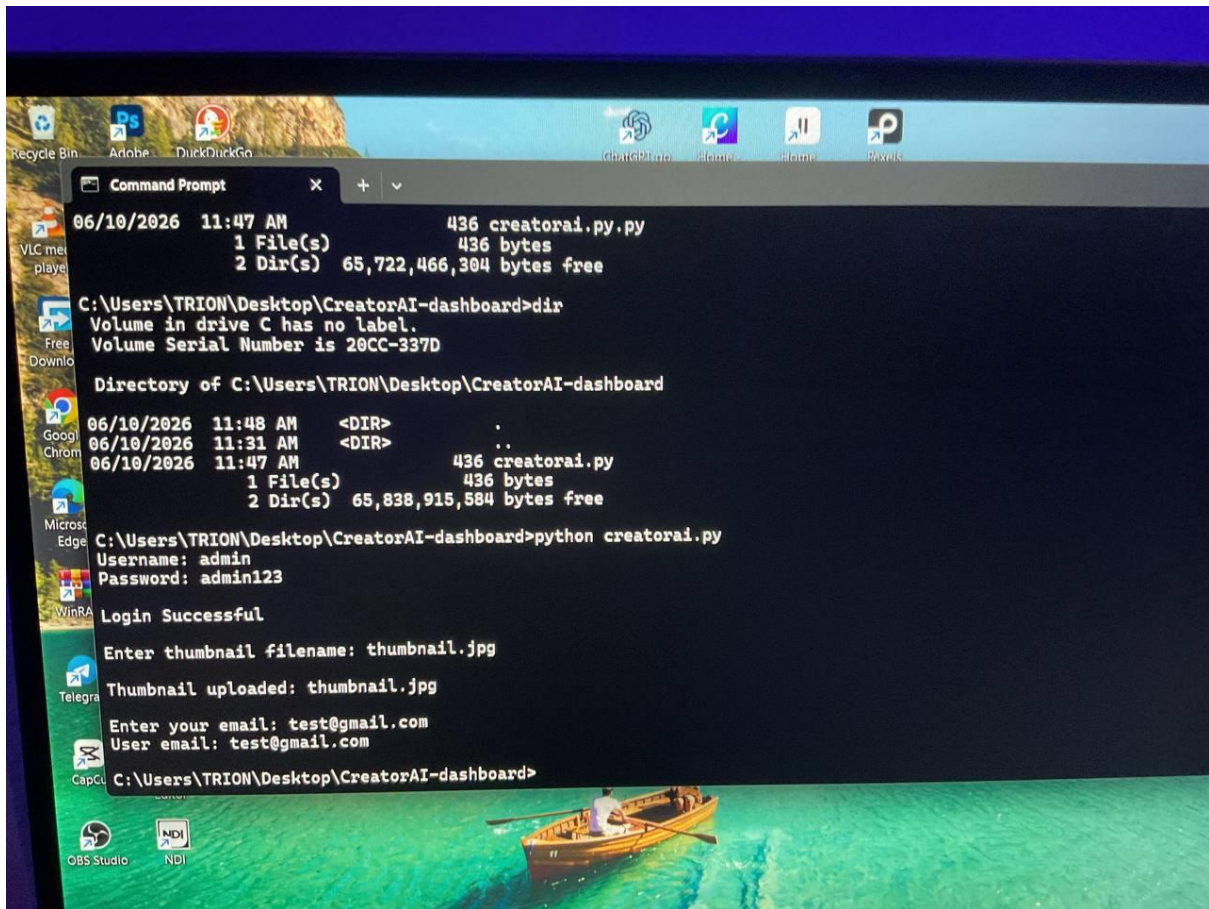


Figure 1: Successful execution of CreatorAI Dashboard

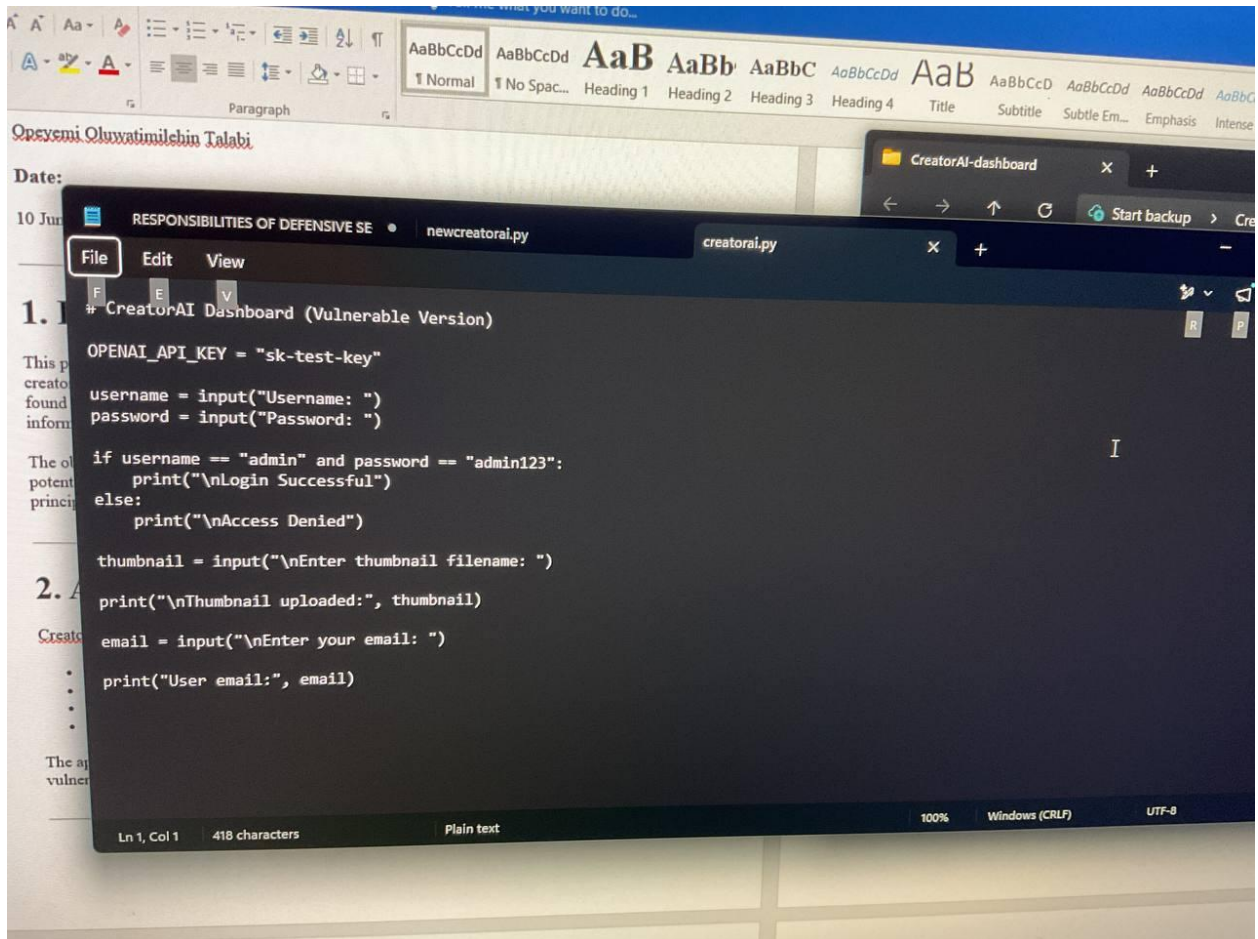


Figure 2: Source code reviewed during security assessment

3. Security Findings

Finding 1: Hardcoded Credentials

Vulnerable Code

```
if username == "admin" and password == "admin123":
```

Description

The application stores login credentials directly within the source code. Any individual with access to the code can view these credentials and gain unauthorized access.

Risk

Attackers can obtain administrative access by simply reading the source code.

Severity

High

Recommendation

Store user credentials securely in a database and implement password hashing using secure algorithms.

Finding 2: Exposed API Key

Vulnerable Code

```
OPENAI_API_KEY = "sk-test-key"
```

Description

The API key is stored directly in the application source code.

Risk

If the source code is exposed, attackers may steal the API key and abuse paid services.

Severity

High

Recommendation

Store API keys using environment variables or secure secret management solutions.

Finding 3: Unsafe File Upload Validation

Description

The application accepts any file name entered by the user without validating the file type.

Risk

Attackers may upload malicious files disguised as legitimate content.

Severity

High

Recommendation

Restrict uploads to approved file types such as JPG, PNG, and JPEG and validate file extensions before processing.

Finding 4: Unlimited Login Attempts

Description

The application does not limit the number of failed login attempts.

Risk

Attackers may perform brute-force attacks by repeatedly guessing passwords until successful.

Severity

Medium

Recommendation

Implement account lockout mechanisms after multiple failed login attempts.

Finding 5: User Data Exposure

Vulnerable Code

```
print("User email:", email)
```

Description

User email addresses are displayed directly on the screen.

Risk

Sensitive user information may be exposed to unauthorized individuals.

Severity

Medium

Recommendation

Mask sensitive information and avoid displaying personal data unnecessarily.

5. Risk Assessment Table

ID	Vulnerability	Severity
F-01	Hardcoded Credentials	High
F-02	Exposed API Key	High
F-03	Unsafe File Upload	High
F-04	Unlimited Login Attempts	Medium
F-05	User Data Exposure	Medium

5. Conclusion

The security review identified five significant vulnerabilities within the CreatorAI Dashboard application. These weaknesses could lead to unauthorized access, data exposure, abuse of services, and malicious file uploads.

Implementing secure coding practices such as secure credential storage, input validation, access control mechanisms, and proper handling of sensitive information would significantly improve the security posture of the application.

This assessment demonstrates the importance of integrating security considerations throughout the software development lifecycle.